CYBERSECURITY / BLUE TEAM / SOC

Windows Operating System Fundamentals for SOC Analysts

Chapter 4 - Personal SOC Study Notes

Event Logs | Sysmon | Registry | DFIR | Detection

Prepared by Ahmed Hany

SOC Learning Series

Chapter 4 at a Glance

A plain-language map of what matters during a Windows SOC investigation.

1

Processes & Parent-Child

Identify what ran, the parent process, command line, user context, hash, and whether the lineage makes sense.

2

Event Logs & Sysmon

Use Security, System, PowerShell, Defender, and Sysmon telemetry to reconstruct execution, network, file, and Registry activity.

3

Authentication & Active Directory

Correlate 4624/4625, Kerberos events, logon types, privileged groups, NTLM/Kerberos behavior, and remote access.

4

Registry & Persistence

Check Run keys, services, scheduled tasks, WMI, COM, Startup folders, and other mechanisms that survive logon or reboot.

5

Networking & Lateral Movement

Tie SMB, RDP, WinRM, RPC, LDAP, and outbound C2 connections back to the responsible process and account.

6

Windows DFIR Artifacts

Use Prefetch, Amcache, ShimCache, MFT/USN, SRUM, LNK/Jump Lists, Recycle Bin, and PowerShell history as corroborating evidence.

7

Detection & Threat Hunting

Turn Windows artifacts into Sysmon/Sigma/KQL/SIEM detections and repeatable hunting hypotheses rather than one-off checks.

A Repeatable Windows Investigation Workflow

Use the same sequence for suspicious PowerShell, new services, brute-force alerts, credential access, lateral movement, and most endpoint cases.

1 ALERT & CONTEXT

What fired? Record host, user, time, alert source, severity, and the exact triggering evidence.

↓

2 PROCESS

What executed? Check parent/child lineage, command line, signer, hash, integrity level, and user token.

↓

3 IDENTITY & AUTH

Which account/session explains it? Correlate logon type, source host/IP, Kerberos/NTLM events, and privilege changes.

↓

4 NETWORK

What did it communicate with? Tie DNS, remote IPs, SMB/RDP/WinRM, and listening ports back to the owning process.

↓

5 FILES & REGISTRY

What changed? Review dropped files, MFT/USN evidence, Run keys, Registry changes, services, and task definitions.

↓

6 PERSISTENCE & CREDENTIAL ACCESS

How does it survive, and what credentials may be exposed? Sweep services, tasks, WMI/COM, Startup, LSASS access, and ticket abuse.

↓

7 TIMELINE & SCOPE

Put process, logon, file, Registry, and network events in order; then pivot across related hosts, users, and indicators.

↓

8 IOC + RESPONSE

Record hashes, IPs/domains, accounts, paths, service/task names, MITRE mapping, containment actions, and final classification.

Mental model: Baseline -> anomaly -> corroborating evidence -> scope -> response

Contents

Chapter 4: Windows Operating System Fundamentals for SOC Analysts **1**

1. Introduction 1
2. Technical Theory 4
3. Installation 22
4. Commands / Usage — Investigation Command Reference 24
5. Practical Examples 26
6. SOC Investigation — Ten Complete Workflows 27
7. Detection 30
8. Threat Hunting 33
9. Incident Response 35
10. Digital Forensics — Windows DFIR Artifacts Deep Dive 36
11. Common Malware Techniques — Windows Persistence and Attack Techniques 39
12. Blue Team Perspective 42
13. Red Team Perspective 43
14. Real Case Studies 43
15. Common Mistakes 44
16. Interview Questions 45
17. Real Interview Scenarios 48
18. Hands-on Labs 49
19. Cheat Sheet 51
20. Summary 53

Chapter 4: Windows Operating System Fundamentals for SOC Analysts

> *"Windows gives you more telemetry than any other OS — Event Logs, Sysmon, Registry, Prefetch, Amcache. The job isn't finding evidence. It's knowing which of the dozens of sources actually has it."*

---

1. Introduction

1.1 Why Windows Matters to a SOC Analyst

Windows remains the dominant operating system across enterprise endpoints and a substantial share of servers, meaning the majority of user-facing compromises — phishing execution, credential theft, ransomware deployment, lateral movement via Active Directory — occur on Windows systems. Unlike Linux, where visibility often depends on the analyst assembling their own toolkit (Chapter 3), Windows ships with an extraordinarily rich native telemetry surface: Event Logs, the Registry, Prefetch, Amcache, ShimCache, and (when deployed) Sysmon. The SOC challenge with Windows is rarely "is there evidence?" — it's "which of the dozens of available artifact sources holds the evidence I need, and how do I interpret it

correctly?”

1.2 Where Windows Knowledge Is Used in a SOC

Area	Application
Endpoint Investigation	Process trees, Event Logs, Sysmon telemetry for workstation/server compromise
Active Directory Security	Kerberos/NTLM authentication abuse, lateral movement, privilege escalation across a domain
DFIR	Prefetch/Amcache/ShimCache for execution evidence, Registry for persistence, MFT/USN Journal for file activity
Threat Hunting	LOLBins abuse, PowerShell attack patterns, persistence mechanism sweeps
Detection Engineering	Sigma/Sysmon/Windows Event Log-based detections, KQL for Microsoft Defender/Sentinel

1.3 Real SOC Use Cases

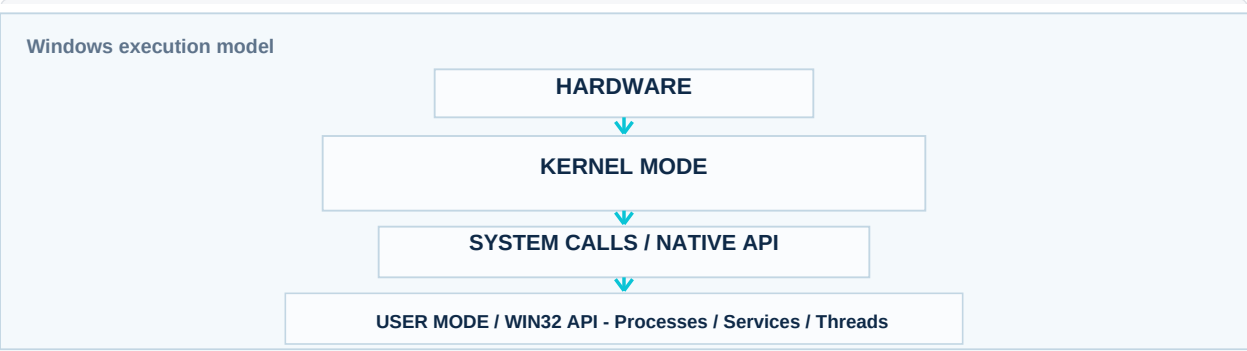
1. A phishing email delivers a macro-enabled document that spawns `powershell.exe` with an encoded command — the SOC must decode it, trace the resulting process tree, and determine the payload’s behavior.
2. An analyst investigates repeated Event ID 4625 (failed logon) followed by a 4624 (success) against a domain controller, confirming a successful brute-force/password-spray attack.
3. A threat hunter sweeps for new Registry Run key entries across the fleet, catching a persistence mechanism before it’s used to redeploy a dropped payload.
4. A DFIR investigator uses Prefetch and Amcache to prove a specific tool executed on a host, even though the attacker deleted the binary and cleared some Event Logs.

1.4 Common Misconceptions

Misconception	Reality
“Antivirus/Defender alone is sufficient visibility”	Modern intrusions routinely use LOLBins and fileless techniques designed specifically to evade signature-based AV; Event Logs, Sysmon, and behavioral detection are essential complements
“Deleting a file removes all evidence it ran”	Prefetch, Amcache, ShimCache, and Event Logs can independently prove execution occurred even after the binary itself is deleted

Misconception	Reality
“The Security log captures everything important”	Sysmon captures process/network/file detail the built-in Security log does not; PowerShell-specific logs capture script content the Security log never sees
“UAC prevents privilege escalation”	UAC is a convenience/consent boundary, not a hard security boundary — numerous well-documented bypass techniques exist

1.5 Architecture Overview



Windows separates kernel mode from user mode. Applications normally execute in user mode and reach privileged OS resources through controlled Windows API and system-call layers. For a SOC analyst, this boundary helps explain why process lineage, access tokens, handles, loaded modules, drivers, and kernel activity provide different types of evidence. The same core model exists in Linux (Chapter 3), but Windows exposes it through Windows-specific telemetry such as Event Logs, Sysmon, the Registry, and DFIR artifacts.

SOC ANALYST FOCUS

For day-to-day Windows SOC work, keep five questions in mind:

1. What process ran, who spawned it, and with what command line?
2. What user, logon session, or authentication event explains the activity?
3. What files, Registry keys, services, or scheduled tasks changed?
4. What network connections or remote-execution activity followed?
5. What persistence, credential access, or lateral movement still needs scoping?

This keeps the chapter focused on investigation, evidence correlation, and detection.

1.6 Advantages and Disadvantages (Defensive Perspective)

Aspect	Advantage	Disadvantage
Rich native telemetry	Event Logs, Registry, Prefetch, Amcache all ship built-in	Requires deep specific knowledge to interpret correctly; volume can be overwhelming without tuning
Active Directory centralization	Centralized authentication/policy enables strong enterprise-wide control	Also centralizes risk — AD compromise often means full domain compromise
Enormous LOLBin surface	Legitimate admin tools (PowerShell, WMI, `rundll32`) enable powerful administration	The exact same tools are the most common vector for attacker Living-off-the-Land techniques
UAC / mandatory integrity control	Reduces casual privilege misuse, prompts for consent	Widely known bypass techniques mean it should not be relied upon as a hard security boundary

---

2. Technical Theory

2.1 User Mode vs. Kernel Mode

Aspect	Kernel Mode	User Mode
Privilege	Full hardware/memory access	Restricted, isolated per-process memory
Components	NT kernel, HAL, device drivers, the Executive	Applications, most services, subsystem DLLs
Crash impact	A kernel-mode fault causes a full system crash (BSOD)	A user-mode fault typically crashes only that process
Security relevance	Kernel-mode exploits (driver vulnerabilities, rootkits) grant total system control	Most malware initially executes here; privilege escalation to SYSTEM/kernel is a key attacker objective

2.2 Processes, Threads, and Handles

A **process** is an instance of a running program with its own virtual address space; a **thread** is a schedulable unit of execution within a process. A **handle** is a reference a process holds to a kernel object (a file, registry key, process, thread, or other resource) — analogous to a Linux file descriptor but covering a much broader

range of object types. Handles are enumerable per-process and are a key forensic/live-investigation artifact (via tools like Sysinternals `Handle`/`Process Explorer`), revealing exactly which files, registry keys, or other processes a given process currently has open.

2.3 Tokens and Sessions

An **access token** represents a process or thread's security context — the user's SID, group memberships, and privileges (e.g., `SeDebugPrivilege`, `SeImpersonatePrivilege`). Tokens are central to several major attack techniques: **token impersonation/theft** (stealing a higher-privileged process's token to act as that user without needing their password), and privilege escalation techniques that abuse specific privileges like `SeImpersonatePrivilege` (e.g., the "Potato" family of exploits).

A **session** represents a logon instance (Session 0 is reserved for services since Windows Vista, isolating them from interactive user sessions for security reasons; interactive user logons occur in Session 1+).

2.4 DLLs, EXEs, and the Windows API

A **DLL** (Dynamic Link Library) contains code/data that can be loaded and shared by multiple processes at runtime; an **EXE** is a standalone executable. The **Windows API** (Win32 API) is the primary interface user-mode applications use to request OS services (file operations, process creation, networking) — implemented across core DLLs like `kernel32.dll`, `advapi32.dll`, and `ntdll.dll` (which itself wraps the lower-level Native API/system calls). Attackers extensively target DLL loading behavior: **DLL hijacking/search order hijacking** (Section 11), **DLL injection**, and **process injection** techniques all manipulate how and what code gets loaded into a process's memory space.

2.5 Users, Groups, Local vs. Domain Accounts, and UAC

Concept	Explanation
Local account	Exists only on a single machine's local SAM (Security Account Manager) database
Domain account	Exists in Active Directory, usable across any domain-joined machine, centrally managed
UAC (User Account Control)	Requires explicit consent/credentials to elevate a process to administrator, even for an admin-group user, via a **split token** model (a standard-user token for daily use, an elevated token available on consent)

Concept	Explanation
Privileges	Fine-grained rights (`SeDebugPrivilege`, `SeBackupPrivilege`, `SeImpersonatePrivilege`) assignable independently of group membership, several of which are well-documented privilege escalation vectors when improperly granted

Why UAC is not a hard security boundary: Microsoft has explicitly stated UAC is a convenience feature, not a security boundary — numerous "UAC bypass" techniques (auto-elevating binaries, DLL hijacking of an auto-elevating process, COM elevation moniker abuse) allow a standard admin-group user's malware to silently reach a full elevated token without a consent prompt ever appearing (Section 11).

2.6 The Windows Filesystem — NTFS Deep Dive

NTFS (NT File System) is the default modern Windows filesystem, replacing the older **FAT32** (still used for some removable media/compatibility scenarios, lacking NTFS's permissions, journaling, and advanced metadata features).

NTFS Feature	Forensic/Security Relevance
MFT (Master File Table)	The central NTFS metadata structure — every file/directory has an MFT record containing timestamps, permissions, and (for small files) even the file content itself; the single most important structure for Windows file-level forensics
USN Journal (Update Sequence Number Journal / `$UsnJrnl`)	A continuously-updated log of every file system change (create, rename, delete, modify) — invaluable for reconstructing file activity even after the file itself has been deleted, since deletion is itself a logged event
Alternate Data Streams (ADS)	NTFS allows a file to have multiple named data streams beyond its primary content (`file.txt:hidden_stream`); attackers have historically abused ADS to hide payloads within an otherwise innocuous-looking file, since ADS content is invisible in standard directory listings

NTFS Feature	Forensic/Security Relevance
File Metadata (Timestamps)	NTFS tracks Modified, Accessed, Created (Birth), and MFT-Entry-Modified (sometimes abbreviated with Created folded in as “MACE” or “MACB” depending on convention) — timestamp manipulation (“timestomping”) is a recognized anti-forensics technique attackers use to blend malicious file activity with legitimate baseline timestamps

```powershell
# List alternate data streams on a file
Get-Item -Path 'C:\path\to\file.txt' -Stream *

# View a specific ADS
Get-Content -Path 'C:\path\to\file.txt' -Stream hidden_stream
```

2.7 Key Filesystem Locations for SOC Investigation

Location	Relevance
`C:\Windows\System32`	Core OS binaries — trojanized/replaced binaries here are a major red flag
`C:\Program Files`, `C:\Program Files (x86)`	Installed application binaries
`C:\Users\<user>\AppData\Roaming` / `\Local` / `\LocalLow`	Extremely common malware staging/persistence location; per-user application data, frequently used because it doesn’t require administrative privilege to write to
`C:\Windows\Temp`, `%TEMP%`	Common payload staging location, analogous to Linux `/tmp`
`C:\Users\<user>\Desktop/Downloads`	Common initial payload landing location post-phishing-download
`C:\ProgramData`	Hidden-by-default, writable-by-standard-users location frequently abused for malware staging
Startup folders (`...\Microsoft\Windows\Start Menu\Programs\Startup`)	Direct persistence — anything here launches at logon (Section 11)
Recycle Bin (`$Recycle.Bin`)	Deleted file recovery; retains original file metadata even after “deletion”

2.8 The Windows Registry — Architecture Deep Dive

The Registry is a hierarchical database storing OS and application configuration, and — critically for SOC purposes — one of the richest persistence and forensic evidence sources on Windows.

Hive	Full Name	Contents
HKLM	HKEY_LOCAL_MACHINE	System-wide configuration (hardware, installed software, services) — persists across all users
HKCU	HKEY_CURRENT_USER	Configuration for the currently logged-on user (a view into `HKU\<user SID>`)
HKCR	HKEY_CLASSES_ROOT	File association and COM object registration data (merged view of `HKLM\Software\Classes` and `HKCU\Software\Classes`)
HKU	HKEY_USERS	Contains a subkey for every loaded user profile (by SID), including the default profile template
HKCC	HKEY_CURRENT_CONFIG	Current hardware profile configuration (a pointer into HKLM hardware config)

Each hive contains **keys** (folder-like containers) holding **values** (named data entries of types like `REG_SZ` (string), `REG_DWORD` (32-bit number), `REG_BINARY`).

2.9 Registry Persistence Locations

Registry Key	Persistence Mechanism
`HKLM\Software\Microsoft\Windows\CurrentVersion\Run` / `RunOnce`	Executes the specified command at boot for all users (`HKLM`) or once (`RunOnce`)
`HKCU\Software\Microsoft\Windows\CurrentVersion\Run`	Same, but scoped to the current user, executing at that user's logon

Registry Key	Persistence Mechanism
`HKLM\Software\Microsoft\Windows NT\CurrentVersion\Winlogon` (`Shell`, `Userinit`)	Modifying these values can hijack the logon process itself to launch attacker code
`HKLM\Software\Microsoft\Windows NT\CurrentVersion\Windows` (`AppInit_DLLs`)	Forces specified DLLs to load into every process using `user32.dll` — a broad, powerful (and increasingly restricted by newer Windows security features) injection/persistence mechanism
`HKCR\CLSID\{...}\InprocServer32`	COM hijacking — replacing the DLL path a legitimate COM object points to, discussed further in Section 11
`HKLM\System\CurrentControlSet\Services\<name>`	Service configuration — a malicious/modified service definition here achieves persistence via the Service Control Manager

```powershell
# View Run key entries
Get-ItemProperty -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run'
Get-ItemProperty -Path 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run'

# View Winlogon Shell/Userinit values (should be explorer.exe and userinit.exe
 ↪  respectively — any deviation is highly suspicious)
Get-ItemProperty -Path 'HKLM:\SOFTWARE\Microsoft\Windows NT\CurrentVersion\Winlogon'
 ↪  -Name Shell, Userinit
```

Why the Registry matters so much for SOC work: Because so many persistence and configuration mechanisms funnel through it, a well-tuned Registry monitoring/auditing capability (via Sysmon Event ID 13 "Registry value set," covered in Section 7) gives a SOC visibility into a huge fraction of all persistence techniques from a single, centralized detection surface — the closest Windows equivalent to Linux's distributed persistence locations (Chapter 3, Section 8.7), but here largely unified under one queryable structure.

2.10 Important Windows Processes and Their Legitimate Purpose

Process	Legitimate Purpose	Common Attacker Abuse
`svchost.exe`	Generic host process for services running from DLLs rather than standalone EXEs; many instances run simultaneously, each hosting a different service group	Masquerading — malware naming itself `svchost.exe` or injecting into a legitimate instance to blend in; genuine `svchost.exe` should always have a parent of `services.exe` and a `-k` command-line argument
`lsass.exe`	Local Security Authority Subsystem Service — manages authentication, stores credential material in memory	**The single highest-value credential-dumping target on Windows** — tools like Mimikatz read LSASS memory to extract plaintext passwords, hashes, and Kerberos tickets (Section 11)
`winlogon.exe`	Manages the logon UI, secure attention sequence (Ctrl+Alt+Del), and logon/logoff process	Winlogon Registry hijacking (Section 2.9); should have very few, well-understood child processes
`services.exe`	The Service Control Manager (SCM) — starts, stops, and manages all Windows services	Should be a direct child of `wininit.exe`; unexpected parentage is anomalous
`explorer.exe`	The Windows shell (desktop, taskbar, file browser)	Common parent for user-launched malware (a user double-clicking a malicious attachment); explorer.exe itself is rarely the malicious component, but is very frequently the *parent* of one
`rundll32.exe`	Legitimately executes functions exported from a DLL	Extremely common **LOLBin** — used to execute malicious code from a DLL, including remotely-hosted payloads, while appearing as a legitimate system process

Process	Legitimate Purpose	Common Attacker Abuse
`regsvr32.exe`	Legitimately registers/unregisters DLL-based COM components	LOLBin abused for "Squiblydoo"-style attacks — can execute a remotely-hosted scriptlet, bypassing some application whitelisting controls
`mshta.exe`	Executes Microsoft HTML Application (`.hta`) files	LOLBin frequently used to execute malicious script content, often as an initial payload stage from a phishing document
`powershell.exe`	Legitimate, powerful administrative scripting shell	Extremely common attack tooling delivery/execution vector (Section 6, PowerShell Security)
`cmd.exe`	Legitimate command interpreter	Frequently seen as an intermediate parent/child in malicious process chains, especially spawned by Office applications or scripting engines
`wscript.exe` / `cscript.exe`	Legitimate Windows Script Host engines for `.vbs`/`.js` scripts	Common initial-stage execution engine for malicious scripts delivered via phishing/macro documents

The universal detection principle: For every process in this table, the question that matters most is **"does this process's parent make sense?"** `winword.exe` spawning `powershell.exe` is anomalous; `explorer.exe` spawning `powershell.exe` is normal (a user manually opening PowerShell). This single heuristic — validating parent-child relationships against a known-good baseline — catches an enormous fraction of real-world Windows intrusions and underlies the majority of Sysmon/EDR-based detection logic covered in Section 7.

2.11 Windows Services Architecture

Services are long-running background processes managed by the **Service Control Manager (SCM, `services.exe`)**, configured in the Registry under `HKLM\System\CurrentControlSet\Services\<name>` and typically running under a **service account** (`LocalSystem`, `NetworkService`, `LocalService`, or a dedicated domain/local account).

```powershell
# List all services with their state and start mode
Get-Service | Select-Object Name, Status, StartType

# View a service's full configuration including binary path
Get-WmiObject Win32_Service -Filter "Name='ServiceName'" | Select-Object Name,
 ↪  PathName, StartName, StartMode

# Create a new service (legitimate admin usage — same mechanism attackers abuse)
New-Service -Name "ExampleSvc" -BinaryPathName "C:\Path\To\Binary.exe" -StartupType
 ↪  Automatic

# Using sc.exe (classic command-line tool)
sc.exe create ExampleSvc binPath= "C:\Path\To\Binary.exe" start= auto
```

Malicious service persistence: An attacker with administrative privilege creates a new service (or modifies an existing one's `ImagePath`) pointing to their payload, set to start automatically at boot — directly analogous to Linux systemd service persistence (Chapter 3, Section 11.1.2). Windows logs this specifically via **Event ID 7045** (New Service Installed) and **4697** (Service Installed, Security log, if service-related auditing is enabled) — both extremely high-value, relatively low-noise detection signals since legitimate new service creation is comparatively infrequent on a stable production host.

2.12 Scheduled Tasks

The **Task Scheduler** provides Windows-native time/event-triggered execution, configured via the Task Scheduler GUI, `schtasks.exe`, or PowerShell's `ScheduledTasks` module, with definitions stored both in the Registry (`HKLM\Software\Microsoft\Windows NT\CurrentVersion\Schedule\TaskCache`) and as XML files under `C:\Windows\System32\Tasks`.

```powershell
# List all scheduled tasks
Get-ScheduledTask | Select-Object TaskName, State

# View full details of a specific task, including its action (what it runs)
Get-ScheduledTask -TaskName "ExampleTask" | Get-ScheduledTaskInfo
(Get-ScheduledTask -TaskName "ExampleTask").Actions

# Using schtasks.exe (classic command-line tool)
schtasks /query /fo LIST /v
schtasks /create /tn "ExampleTask" /tr "C:\Path\To\Payload.exe" /sc onlogon
```

Why scheduled tasks are a favorite persistence/execution mechanism: Like cron on Linux, they survive reboots, can run with SYSTEM privilege, and — since `schtasks` and the Task Scheduler service are both entirely legitimate, expected parts of Windows — task-based execution can blend into normal administrative noise more easily than dropping and directly running a new standalone binary. **Event ID 4698** (Scheduled Task Created) is the primary detection signal, alongside Sysmon Event ID 1 process creation logs show-

ing `schtasks.exe`/`taskeng.exe`/`svchost.exe -k netsvcs` (Task Scheduler service host) launching an unfamiliar payload.

2.13 Windows Networking

Windows implements standard TCP/IP (Chapter 1) plus several Windows/Active-Directory-specific protocols central to enterprise investigation:

Protocol	Port(s)	SOC Relevance
SMB	445 (modern), 139 (legacy NetBIOS)	File/printer sharing, and the primary lateral movement channel (`PsExec`, admin shares `C$`/`ADMIN$`) — Chapter 1, Section 2.13
RPC	135 + dynamic high ports	Underlies WMI remoting and many Windows management operations; a common lateral movement/remote execution channel
LDAP	389 / 636 (LDAPS)	Active Directory querying — reconnaissance tooling (BloodHound, `AdFind`) heavily relies on this
Kerberos	88	Domain authentication — Chapter 1, Section 2.15; Kerberoasting/Golden-Silver Ticket attacks
RDP	3389	Remote Desktop — top ransomware initial-access/lateral-movement vector
WinRM	5985 (HTTP) / 5986 (HTTPS)	PowerShell Remoting — a common living-off-the-land lateral movement channel (`Enter-PSSession`, `Invoke-Command`)

DNS	53	Name resolution; Active Directory-integrated DNS also reveals domain infrastructure during reconnaissance

```
# View active connections with owning process (PowerShell equivalent of ss/netstat)
Get-NetTCPConnection | Select-Object LocalAddress, LocalPort, RemoteAddress,
 ↪  RemotePort, State, OwningProcess

# Correlate a connection's owning process to its name
Get-Process -Id <OwningProcess>

# View Windows Firewall rules
Get-NetFirewallRule | Where-Object {$_.Enabled -eq 'True'} | Select-Object
 ↪  DisplayName, Direction, Action
```

How network activity appears during a typical attack: Initial C2 beaconing over HTTPS (443) mimicking normal browser traffic; internal reconnaissance via LDAP (389) queries against a domain controller; lateral movement via SMB (445) to admin shares or WinRM (5985) for remote PowerShell execution; and, in ransomware/destructive scenarios, a final burst of SMB traffic as the payload propagates and encrypts file shares across multiple hosts in rapid succession.

2.14 Windows Event Logging Architecture

Windows Event Logs are stored in a structured binary format (`.evtx`) under `C:\Windows\System32\winevt\Logs\`, queryable via Event Viewer, `wevtutil`, PowerShell's `Get-WinEvent`, or forwarded to a SIEM via Windows Event Forwarding (WEF) or an agent.

Log	Contents
Security	Authentication, authorization, object access, privilege use — the primary SOC log, requires audit policy configuration to populate many event types
System	OS/driver/service-level events (service start/stop, system boot)
Application	Application-level events (crashes, application-specific logging)
Windows PowerShell / **Microsoft-Windows-PowerShell/Operational**	PowerShell engine start/stop and (with logging enabled) script block content

Log	Contents
Microsoft-Windows-Windows Defender/Operational	Defender detection, scan, and configuration events
Microsoft-Windows-TaskScheduler/Operational	Scheduled task creation, execution, and modification
Sysmon (Microsoft-Windows-Sysmon/Operational)	Not built-in — requires installing the free Sysinternals Sysmon tool, but provides by far the richest process/network/file/registry telemetry available on Windows

```
# Query the Security log for a specific Event ID
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625} -MaxEvents 50

# Query Sysmon logs for process creation events
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1}
 ↪  -MaxEvents 50

# Export a log for offline analysis
wevtutil epl Security C:\evidence\security.evtx
```

2.15 Critical Windows Event IDs — Full Reference

For each Event ID: meaning, why the SOC cares, a normal example, a suspicious example, investigation steps, detection opportunities, and MITRE ATT&CK mapping.

Event ID 4624 — Successful Logon

- **Meaning:** An account successfully logged on.
- **Why SOC Cares:** Establishes exactly who authenticated, when, how (Logon Type), and from where — foundational for nearly every investigation involving account activity.
- **Normal Example:** Logon Type 2 (Interactive) at a user's own workstation during business hours.
- **Suspicious Example:** Logon Type 3 (Network) or Type 10 (RemoteInteractive/RDP) from an unfamiliar internal host or at an unusual hour, especially for a privileged account.
- **Investigation:** Check the Logon Type, source IP/workstation name, and account; cross-reference against the account's normal behavior baseline; check for a preceding 4625 burst suggesting a successful brute force.
- **Detection Opportunities:** Anomalous Logon Type/source combinations; privileged account logons from non-admin workstations.
- **MITRE ATT&CK:** T1078 (Valid Accounts).

Event ID 4625 — Failed Logon

- **Meaning:** An account failed to log on.
- **Why SOC Cares:** The primary brute-force/password-spray detection signal.
- **Normal Example:** A single failure from a user who mistyped their password, followed by a success.
- **Suspicious Example:** Dozens/hundreds of failures against one or many accounts from a single source in a short window.
- **Investigation:** Aggregate by source IP and target account; check the Failure Reason sub-status code; determine whether any subsequent 4624 success followed.
- **Detection Opportunities:** Threshold-based alerting on failure count per source/account/time window.
- **MITRE ATT&CK:** T1110 (Brute Force), T1110.003 (Password Spraying).

Event ID 4634 — Logoff

- **Meaning:** An account logged off.
- **Why SOC Cares:** Establishes session duration when paired with the corresponding 4624; unusually short or absent logoff events can indicate an abnormal session termination.
- **Investigation:** Pair with the corresponding 4624 by Logon ID to establish session duration.
- **MITRE ATT&CK:** N/A (contextual/supporting event).

Event ID 4648 — Logon Using Explicit Credentials

- **Meaning:** A process attempted to log on using explicitly specified credentials (different from the calling user's own).
- **Why SOC Cares:** Frequently seen with `runas`, scheduled tasks running as another user, and — notably — is a signature of some lateral movement tooling that supplies credentials directly rather than relying on the current session's token.
- **Suspicious Example:** A user's workstation generating this event targeting a remote server using a privileged account's credentials, outside any known administrative workflow.
- **Investigation:** Identify the target account and destination system; correlate with the source process.
- **MITRE ATT&CK:** T1078, T1021 (Remote Services).

Event ID 4672 — Special Privileges Assigned to New Logon

- **Meaning:** A logon was granted one or more sensitive privileges (e.g., `SeDebugPrivilege`, `SeBackupPrivilege`) — typically accompanies an administrator-level logon.
- **Why SOC Cares:** A high volume of unexpected 4672 events, especially correlated with unfamiliar accounts, suggests unauthorized privilege elevation.
- **Investigation:** Correlate the Logon ID with the corresponding 4624 to establish full logon context.
- **MITRE ATT&CK:** T1078.002/.003 (Valid Accounts: Domain/Local Accounts, privileged).

Event ID 4688 — A New Process Has Been Created

- **Meaning:** Process creation — the Security log's version of Sysmon Event ID 1 (less detail by default, but available without third-party tooling if process creation auditing and command-line logging are enabled).

- **Why SOC Cares:** The foundational event for process-tree-based investigation on hosts without Sysmon.
- **Normal Example:** `explorer.exe` spawning a user-launched application.
- **Suspicious Example:** `winword.exe` spawning `powershell.exe` with an encoded command-line argument.
- **Investigation:** Examine parent/child relationship, full command line (requires `Include command line in process creation events` policy enabled), and the process's subsequent behavior.
- **Detection Opportunities:** Parent-child anomaly detection (Section 2.10); suspicious command-line patterns.
- **MITRE ATT&CK:** T1059 (Command and Scripting Interpreter) and virtually any execution-stage technique.

Event ID 4697 — A Service Was Installed in the System (Security Log)

- **Meaning:** A new service was installed, logged to the Security log (requires appropriate audit policy).
- **Why SOC Cares:** Service creation is a top persistence and lateral-movement-execution mechanism (e.g., PsExec creates a temporary service to execute commands remotely).
- **Investigation:** Check the service's binary path, account, and start type; cross-reference with 7045 (System log equivalent).
- **MITRE ATT&CK:** T1543.003 (Create or Modify System Process: Windows Service).

Event ID 4698 — A Scheduled Task Was Created

- **Meaning:** A new scheduled task was created.
- **Why SOC Cares:** A primary persistence mechanism (Section 2.12).
- **Investigation:** Extract the task's action/command from the event's embedded XML; correlate creation time with the responsible process/session.
- **MITRE ATT&CK:** T1053.005 (Scheduled Task).

Event ID 4720 — A User Account Was Created

- **Meaning:** New local or domain account creation.
- **Why SOC Cares:** A classic persistence technique — an attacker with sufficient privilege creates a new account for durable, independent access.
- **Investigation:** Verify against change management records; check for immediate privileged group addition (4728/4732, below) following creation.
- **MITRE ATT&CK:** T1136 (Create Account).

Event ID 4728 / 4732 — Member Added to a Security-Enabled Global/Local Group

- **Meaning:** An account was added to a group (4728 = global group, e.g., Domain Admins; 4732 = local group, e.g., local Administrators).
- **Why SOC Cares:** Privilege escalation via group membership is one of the most direct paths to elevated access in Active Directory environments.

- **Suspicious Example:** An account added to Domain Admins outside of a documented change window, especially shortly after that account's own creation or compromise indicators.
- **Investigation:** Identify who performed the addition (the acting account), verify against change management, and assess what access the new membership grants.
- **MITRE ATT&CK:** T1098 (Account Manipulation).

Event ID 4740 — A User Account Was Locked Out

- **Meaning:** An account exceeded the failed logon threshold and was locked.
- **Why SOC Cares:** Strong corroborating evidence of an ongoing brute-force/password-spray attack.
- **MITRE ATT&CK:** T1110.

Event ID 4768 — A Kerberos Authentication Ticket (TGT) Was Requested

- **Meaning:** Initial Kerberos authentication (AS-REQ/AS-REP exchange, Chapter 1 Section 2.15).
- **Why SOC Cares:** Baseline authentication visibility; also relevant to AS-REP Roasting detection (requests for accounts with pre-authentication disabled show a specific encryption/pre-auth pattern).
- **MITRE ATT&CK:** T1558.004 (AS-REP Roasting).

Event ID 4769 — A Kerberos Service Ticket Was Requested

- **Meaning:** TGS-REQ/TGS-REP exchange for access to a specific service.
- **Why SOC Cares:** The primary Kerberoasting detection signal — a burst of 4769 events for many distinct SPNs from one account, especially using weaker RC4 encryption, is a strong indicator (Chapter 1, Section 7.6).
- **MITRE ATT&CK:** T1558.003 (Kerberoasting).

Event ID 4771 — Kerberos Pre-Authentication Failed

- **Meaning:** A Kerberos pre-authentication attempt failed (commonly due to a bad password).
- **Why SOC Cares:** Another brute-force indicator, specific to Kerberos rather than NTLM-based authentication paths.
- **MITRE ATT&CK:** T1110.

Event ID 7045 — A Service Was Installed on the System (System Log)

- **Meaning:** New service installation, logged to the System log (populated by default, unlike 4697 which requires specific audit policy).
- **Why SOC Cares:** Often the single most reliable, always-on new-service detection signal since it requires no special audit policy configuration.
- **MITRE ATT&CK:** T1543.003.

Event ID 1102 — The Audit Log Was Cleared

- **Meaning:** Someone cleared the Security event log.

- **Why SOC Cares:** One of the strongest possible anti-forensics/defense-evasion indicators — legitimate administrators very rarely clear logs, and an attacker covering their tracks frequently does so immediately after achieving their objective.
- **Investigation:** Treat as a near-certain true positive requiring immediate escalation; identify the account/session that performed the clear and everything else that account touched.
- **MITRE ATT&CK:** T1070.001 (Indicator Removal: Clear Windows Event Logs).

2.16 PowerShell Security Deep Dive

PowerShell is Windows' primary modern administrative scripting shell, built on the .NET framework — and simultaneously the single most heavily abused Living-off-the-Land tool in modern Windows intrusions, since it's pre-installed, extraordinarily capable, and can execute entirely in memory without ever writing a traditional executable to disk.

Execution and Encoded Commands:

```
# Standard script execution
powershell.exe -File script.ps1

# Encoded command (Base64-encoded UTF-16LE) — extremely common in malicious usage
# because it evades naive string-based detection on the raw command line and
# easily carries special characters/newlines
powershell.exe -EncodedCommand <Base64String>

# Decoding an observed encoded command for analysis
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String(
↪  "<Base64String>"))
```

PowerShell Logging Capabilities:

Logging Type	What It Captures	Where
Module Logging	Logs pipeline execution details for specified modules	Event ID 4103, `Windows PowerShell` log
Script Block Logging	Logs the full de-obfuscated content of executed script blocks, including dynamically generated/invoked code	Event ID 4104, `Microsoft-Windows-PowerShell/Operational` log — **the single highest-value PowerShell detection/investigation source**
Transcription	Writes a full text transcript of a PowerShell session to a file	Configurable output directory

Logging Type	What It Captures	Where
AMSI (Antimalware Scan Interface)	Allows AV/EDR engines to inspect script content (including deobfuscated, dynamically-generated code) before execution, not just the static file on disk	Integrated with Defender/EDR products; can be logged via Event ID 4104 as well when content is submitted for scanning

```
# Enable Script Block Logging via Group Policy path equivalent (registry)
Set-ItemProperty -Path
 ↪  'HKLM:\Software\Policies\Microsoft\Windows\PowerShell\ScriptBlockLogging' -Name
 ↪  'EnableScriptBlockLogging' -Value 1
```

Constrained Language Mode: Restricts PowerShell to a safe subset of the language (blocking direct .NET method invocation, COM object creation, and other capabilities commonly abused by offensive tooling) — typically enforced via AppLocker/WDAC policy for non-administrative contexts, substantially reducing the attack surface available to a compromised standard-user session.

Common PowerShell Attack Techniques:

Technique	Description
Encoded/obfuscated one-liners	Evading naive signature and string-matching detection
In-memory/reflective execution (`Invoke-Expression`, `IEX (New-Object Net.WebClient).DownloadString(...)`)	Fetching and executing a remote payload entirely in memory, never touching disk
AMSI bypass techniques	Patching or otherwise disabling the in-process AMSI scanning interface before executing malicious content
Credential access modules (e.g., PowerSploit/Mimikatz-wrapping scripts)	Using PowerShell as the delivery/execution wrapper for credential theft tooling
WMI/PowerShell Remoting for lateral movement	`Invoke-Command`/`Enter-PSSession` against remote hosts using stolen or legitimate credentials

Detection and Investigation: Script Block Logging (Event ID 4104) is the single most valuable data source — it reveals the actual de-obfuscated script content regardless of how heavily the original command line was obfuscated or encoded, since PowerShell must eventually resolve the code to its real form to execute

it. Investigators should always pull the full 4104 content for any suspicious PowerShell activity rather than relying on the (potentially heavily obfuscated) raw command line alone.

2.17 Windows Authentication — NTLM, Kerberos, and Active Directory Fundamentals

Concept	Explanation
NTLM	A legacy challenge-response authentication protocol, still used as a fallback when Kerberos isn't available (e.g., workgroup/non-domain scenarios, IP-based access, some legacy applications); vulnerable to relay and hash-based attacks (Pass-the-Hash, Section 11)
Kerberos	The default, more secure Active Directory authentication protocol (Chapter 1, Section 2.15) — ticket-based, avoiding transmission of the password/hash itself for each authentication
LDAP	The protocol used to query and modify Active Directory's directory data
Active Directory (AD)	Microsoft's centralized directory service — stores users, computers, groups, and policy, managed by **Domain Controllers (DCs)**, which are themselves high-value targets since full DC compromise typically means full domain compromise
Group Policy	Centralized configuration management pushed from AD to domain-joined machines — can be abused by an attacker with sufficient privilege to push malicious configuration/scripts fleet-wide
Access Tokens	Covered in Section 2.3 — the runtime representation of an authenticated session's security context
Credential Storage	LSASS memory (Section 2.10), the SAM database (local accounts), NTDS.dit (the AD database on domain controllers, containing every domain account's password hash)

Suspicious authentication patterns to hunt for (Windows-specific, extending Chapter 3's Linux equivalents):

Pattern	Likely Indicator
NTLM authentication where Kerberos would be expected	Possible downgrade attack or relay attempt
A single account authenticating from many distinct hosts in a short window	Possible Pass-the-Hash/Pass-the-Ticket lateral movement using stolen credential material
4769 burst with RC4 encryption across many SPNs	Kerberoasting
4768 for accounts with pre-authentication disabled	AS-REP Roasting
Privileged group membership change outside a change window	Account manipulation/privilege escalation

---

3. Installation

This section covers setting up the essential SOC toolkit for Windows investigation: Sysmon (for rich telemetry), PowerShell logging, and Sysinternals utilities.

3.1 Installing and Configuring Sysmon

```
# Download Sysmon (Sysinternals) and a well-maintained community configuration
# (e.g., the widely-used SwiftOnSecurity or Olaf Hartong configs)
.\Sysmon64.exe -i sysmonconfig.xml

# Verify installation
Get-Service Sysmon64
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'}
 ↪  -MaxEvents 5

# Update configuration on an already-installed instance
.\Sysmon64.exe -c sysmonconfig-updated.xml
```

Key Sysmon Event IDs for SOC purposes:

Event ID	Name	Value
1	Process Creation	Full command line, parent/child, hashes — the single most valuable Sysmon event
2	File Creation Time Changed	Timestomping detection
3	Network Connection	Source/destination, process, port

Event ID	Name	Value
5	Process Terminated	Process lifecycle tracking
6	Driver Loaded	Kernel driver load events — rootkit/EDR-evasion relevant
7	Image Loaded	DLL load events — DLL hijacking/injection detection
8	CreateRemoteThread	Classic process injection indicator
10	Process Access	LSASS access attempts (credential dumping detection)
11	File Create	File creation/modification, useful for payload staging detection
12/13/14	Registry Events	Registry persistence detection
22	DNS Query	Process-to-DNS-query correlation

3.2 Enabling Key Audit Policies (Required for Many Event IDs to Populate)

```
# Enable command-line auditing for process creation events (4688)
auditpol /set /subcategory:"Process Creation" /success:enable

# Include command line arguments in 4688 events (Registry setting)
Set-ItemProperty -Path
 ↪  'HKLM:\SOFTWARE\Microsoft\Windows\CurrentVersion\Policies\System\Audit' -Name
 ↪  'ProcessCreationIncludeCmdLine_Enabled' -Value 1

# Enable logon/logoff and account management auditing categories
auditpol /set /category:"Logon/Logoff" /success:enable /failure:enable
auditpol /set /category:"Account Management" /success:enable /failure:enable

# Enable PowerShell Script Block Logging (Section 2.16)
# via Group Policy or direct registry key as shown previously
```

3.3 Essential Sysinternals Tools

Tool	Purpose
Process Explorer	Rich, real-time process tree/handle/DLL viewer, far more detailed than Task Manager
Autoruns	Enumerates virtually every persistence location (Run keys, services, scheduled tasks, WMI, etc.) in one unified view
TCPView	Live network connection viewer with owning process
Handle	Command-line handle enumeration
Strings	Extract printable strings from a binary (Windows equivalent of the Linux `strings` command, Chapter 3 Section 4.4)
PsExec	Legitimate remote execution tool — also one of the most commonly abused lateral movement tools when used by an attacker with valid credentials

3.4 Configuration Best Practices

- **Deploy Sysmon fleet-wide with a well-maintained, community-vetted configuration**, tuned to exclude known-noisy legitimate activity rather than a default/unfiltered configuration.
- **Enable command-line process auditing** — without it, 4688 events show that a process ran but not what arguments it was given, dramatically reducing investigative value.
- **Forward all logs off-host immediately** (Windows Event Forwarding or a SIEM agent) — exactly as with Linux (Chapter 3), an attacker with sufficient privilege can clear local logs (Event ID 1102).
- **Enable PowerShell Script Block Logging fleet-wide** — the single highest-value PowerShell-specific data source, given how heavily PowerShell is abused in modern intrusions.

---

4. Commands / Usage — Investigation Command Reference

4.1 Process Investigation

```
# List processes with parent PID and command line
Get-CimInstance Win32_Process | Select-Object ProcessId, ParentProcessId, Name,
 ↪  CommandLine

# Process tree view (requires Sysinternals or a script; native PowerShell lacks a
 ↪  built-in tree view)
Get-Process | Select-Object Id, ProcessName, Path

# Find a specific process by name
Get-Process -Name powershell

# Kill a process
Stop-Process -Id <pid> -Force
```

4.2 Registry Investigation

```
# Query a specific key
Get-ItemProperty -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run'

# Search the Registry for a specific string (e.g., a suspicious file path) across a hive
Get-ChildItem -Path 'HKLM:\Software' -Recurse -ErrorAction SilentlyContinue |
  Where-Object { (Get-ItemProperty $_.PSPath -ErrorAction SilentlyContinue) -match
   ↪  'suspicious_string' }
```

4.3 Event Log Querying

```
# Query by ID, time range, and specific fields
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624,4625;
 ↪  StartTime=(Get-Date).AddHours(-24)} |
  Select-Object TimeCreated, Id, @{n='Account';e={$_.Properties[5].Value}}

# Export for offline analysis
wevtutil epl Security C:\evidence\security_export.evtx /q:"*[System[(EventID=4624 or
 ↪  EventID=4625)]]"
```

4.4 Network Investigation

```
Get-NetTCPConnection -State Established | Select-Object LocalAddress, RemoteAddress,
 ↪  RemotePort, OwningProcess
Get-Process -Id (Get-NetTCPConnection -RemoteAddress <suspect_ip>).OwningProcess
```

4.5 Common Mistakes in Command Usage

Mistake	Why It Happens	Fix
Relying on 4688 without command-line auditing enabled	Assuming default configuration is sufficient	Explicitly enable command-line inclusion (Section 3.2) fleet-wide before an incident, not during one
Ignoring Sysmon in favor of Security log alone	Not realizing the depth gap between the two	Deploy Sysmon wherever possible; it captures network, DLL load, and registry detail the Security log does not
Treating the raw PowerShell command line as the full picture	Not accounting for encoding/obfuscation	Always pull Event ID 4104 (Script Block Logging) for the de-obfuscated actual content

---

5. Practical Examples

5.1 Simple: Decoding a Suspicious PowerShell EncodedCommand

```
$encoded = "SQBFAFgA-
 ↪  IAAoAE4AZQB3AC0ATwBiAGoAZQBjAHQAIABOAGUAdAAuAFcAZQBiAEMAbABpAGUAbgB0ACkALgBEAG8AdwBuAGwAbwBhAGQAUw
[System.Text.Encoding]::Unicode.GetString([System.Convert]::FromBase64String($encoded
 ↪  ))
```

5.2 Intermediate: Identifying a Suspicious Parent-Child Process Relationship

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=1}
 ↪  -MaxEvents 500 |
  ForEach-Object {
    $xml = [xml]$_.ToXml()
    [PSCustomObject]@{
      Time = $_.TimeCreated
      Image = $xml.Event.EventData.Data[4].'#text'
      ParentImage = $xml.Event.EventData.Data[19].'#text'
    }
  } | Where-Object { $_.ParentImage -match 'winword|excel|outlook' -and $_.Image -match
   ↪  'powershell|cmd' }
```

5.3 Advanced: Extracting Full Script Block Content for a Suspicious PowerShell Session

```
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational';
↪  Id=4104} -MaxEvents 100 |
  Where-Object { $_.Message -match 'DownloadString|IEX|Invoke-Expression' } |
  Select-Object TimeCreated, Message
```

5.4 Enterprise: Fleet-Wide New-Service Baseline Monitoring

Centrally collect Event ID 7045 (or 4697) from every host via Windows Event Forwarding, alert on any new service whose binary path doesn't match a pre-approved software deployment list, and route matches to a SOC queue for immediate triage — directly analogous to the Linux SUID/cron baseline diffing approach from Chapter 3.

5.5 Real-World: Correlating Kerberoasting Across Sysmon, Security Log, and Network Capture

Combine Event ID 4769 (Security log, RC4 encryption, many SPNs from one account) with Sysmon Event ID 1 (identifying the exact tool/process that issued the requests) and, where available, a Wireshark/tcpdump capture on the domain controller segment (Chapters 2-3) filtered to `kerberos.msgtype == 12` — providing three independent, corroborating evidence sources for a single confirmed finding.

---

6. SOC Investigation — Ten Complete Workflows

Each workflow follows: **Alert → Initial Triage → Logs → Process Tree → Network → Artifacts → IOC Extraction → MITRE ATT&CK → Root Cause → Containment → Eradication → Recovery → Final Report.**

6.1 Suspicious PowerShell

- **Alert:** EDR/Sysmon flags `powershell.exe` with an encoded command line.
- **Initial Triage:** Decode the command (Section 5.1); assess whether it downloads/executes remote content.
- **Logs:** Pull Event ID 4104 (Script Block Logging) for full de-obfuscated content; Event ID 4688/Sysmon 1 for process context.
- **Process Tree:** Identify the parent (Office app? scripting engine? legitimate admin session?).
- **Network:** Check Sysmon Event ID 3/22 for any connection/DNS query the script triggered.
- **Artifacts:** Any dropped files, registry modifications, or scheduled tasks created by the script.
- **IOC Extraction:** C2 domain/IP, script hash, any unique strings.
- **MITRE ATT&CK:** T1059.001 (PowerShell).
- **Root Cause:** Typically a phishing document macro or a malicious download.
- **Containment:** Isolate the host; block the identified C2 indicator.
- **Eradication:** Remove any persistence the script established; rotate any credentials it may have accessed.

- **Recovery:** Re-image or validate clean state before returning to production.
- **Final Report:** Full timeline from initial execution through containment, with MITRE mapping and remediation recommendations.

6.2 Suspicious Process

- **Alert:** An unfamiliar or LOLBin process (Section 2.10) observed with anomalous arguments/parentage.
- **Initial Triage:** Confirm process identity, hash, and signing status.
- **Logs:** Sysmon Event ID 1 for full command line and lineage; Event ID 4688 as a fallback.
- **Process Tree:** Full ancestor chain to PID 1 equivalent (or the initial user session).
- **Network:** Any connections spawned by the process (Sysmon 3).
- **Artifacts:** Files written/modified (Sysmon 11), registry changes (Sysmon 12/13).
- **IOC Extraction:** File hash, C2 indicators, persistence artifacts.
- **MITRE ATT&CK:** Varies by technique (commonly T1218 Signed Binary Proxy Execution for LOLBins).
- **Root Cause → Containment → Eradication → Recovery → Report:** As above, tailored to findings.

6.3 New Service

- **Alert:** Event ID 7045/4697 for an unfamiliar service.
- **Initial Triage:** Check the service's binary path, account, and start type against approved software.
- **Logs:** 7045/4697 plus Sysmon Event ID 1 for the process that created the service.
- **Process Tree:** What process/session created the service (often `services.exe` itself, with the *originating* session identified via the correlated process creation event).
- **Network:** Any subsequent connections from the new service's process.
- **Artifacts:** The service binary itself, for hash/static analysis.
- **IOC Extraction:** Service name, binary path/hash, account used.
- **MITRE ATT&CK:** T1543.003.
- **Root Cause → Containment → Eradication → Recovery → Report:** Stop/delete the malicious service, investigate how the creating account/session was obtained.

6.4 Scheduled Task

- Follows the same pattern as 6.3, keyed on Event ID 4698, with the task's XML action definition as the primary artifact and `schtasks.exe`/Task Scheduler service as the typical creating process context. **MITRE ATT&CK:** T1053.005.

6.5 Brute Force

- **Alert:** High volume of 4625 (or 4771 for Kerberos) against one or many accounts.
- **Initial Triage:** Aggregate by source IP and target account to distinguish brute force (one account, many attempts) from spraying (many accounts, few attempts each).
- **Logs:** 4625/4771 for failures, 4624/4768 for any eventual success, 4740 for any resulting lockouts.
- **Network:** Source IP reputation and geolocation.

- **IOC Extraction:** Source IP(s), targeted account list.
- **MITRE ATT&CK:** T1110/T1110.003.
- **Containment:** Block source IP at the perimeter; force password reset on any successfully-compromised account.

6.6 RDP Attack

- **Alert:** 4625 bursts against RDP (Logon Type 10), or a successful 4624 Type 10 from an unfamiliar source.
- **Logs:** 4625/4624 with Logon Type 10; Event ID 1149 (RDP-specific authentication success, Terminal-Services-RemoteConnectionManager log).
- **Network:** Source IP, session duration.
- **Process Tree:** What the RDP session did once authenticated (Sysmon 1 for the resulting interactive session's process activity).
- **MITRE ATT&CK:** T1021.001 (Remote Services: RDP), T1110 as the likely initial access method.

6.7 Credential Dumping

- **Alert:** Sysmon Event ID 10 (Process Access) showing an unfamiliar process accessing `lsass.exe` with suspicious access rights.
- **Logs:** Sysmon 10 for the LSASS access; Sysmon 1 for the accessing process's full identity/lineage.
- **Process Tree:** Trace the accessing process (often a renamed/disguised credential-dumping tool) back to its origin.
- **IOC Extraction:** Tool hash, any output file it wrote (a dumped LSASS memory file).
- **MITRE ATT&CK:** T1003.001 (OS Credential Dumping: LSASS Memory).
- **Containment:** Isolate immediately — this is a high-severity finding; assume all credentials on the host are compromised.

6.8 Lateral Movement

- **Alert:** SMB/WinRM/RDP connections between workstations (not to expected servers), or PsExec-style service creation on a remote host.
- **Logs:** 4624 Logon Type 3 (network) on the target, 4697/7045 for any service created remotely, Sysmon 1/3 on both source and destination.
- **Network:** SMB (445)/WinRM (5985) traffic between non-server hosts.
- **MITRE ATT&CK:** T1021 (Remote Services family), T1570 (Lateral Tool Transfer).

6.9 Malware Persistence

- **Alert:** New Run key entry, service, scheduled task, or startup folder item (via Autoruns sweep or Sysmon 13).
- **Logs:** Sysmon 12/13 (Registry), 4698/7045, file creation in Startup folder (Sysmon 11).
- **MITRE ATT&CK:** T1547.001 (Registry Run Keys/Startup Folder) and related sub-techniques (Section 11).

6.10 Suspicious Registry Modification

- **Alert:** Sysmon Event ID 13 (Registry value set) on a high-value key (Run keys, Winlogon, AppInit_DLLs).
- **Logs:** Sysmon 12 (key/value create/delete) and 13 (value set) together with the modifying process's Sysmon 1 record.
- **MITRE ATT&CK:** Varies by key (T1547.001 Run Keys, T1546.010 AppInit DLLs, T1546.011 Application Shimming, etc.).

---

7. Detection

7.1 MITRE ATT&CK Mapping — Windows Techniques Summary

Technique	ID	Primary Windows Detection Source
Valid Accounts	T1078	4624/4625 Logon Type analysis
Brute Force / Password Spraying	T1110/.003	4625/4771 aggregation
OS Credential Dumping: LSASS	T1003.001	Sysmon 10 (Process Access to lsass.exe)
Pass-the-Hash / Pass-the-Ticket	T1550.002/.003	4624 Logon Type 3 anomalies, 4769 anomalies
Kerberoasting	T1558.003	4769 RC4 burst
AS-REP Roasting	T1558.004	4768 pre-auth-disabled pattern
PowerShell	T1059.001	Event ID 4104, Sysmon 1
WMI	T1047	Sysmon 19/20/21 (WMI events), 4688 for `wmic.exe`/`wmiprvse.exe` anomalies
Scheduled Task	T1053.005	4698, Sysmon 1
Windows Service	T1543.003	7045/4697
Registry Run Keys / Startup Folder	T1547.001	Sysmon 12/13
DLL Search Order Hijacking	T1574.001	Sysmon 7 (Image Loaded) anomalies
Process Injection	T1055	Sysmon 8 (CreateRemoteThread), Sysmon 10

Technique	ID	Primary Windows Detection Source
Remote Services (SMB/RDP/WinRM)	T1021	4624 Type 3/10, SMB/WinRM traffic patterns
Indicator Removal: Clear Logs	T1070.001	Event ID 1102

7.2 Sigma Rule — LSASS Access from an Unsigned/Unusual Process

```yaml
title: Suspicious Process Access to LSASS Memory
id: b1c2d3e4-0000-4a1b-9c3d-example000010
status: experimental
logsource:
  category: process_access
  product: windows
detection:
  selection:
    TargetImage|endswith: '\lsass.exe'
    GrantedAccess:
      - '0x1010'
      - '0x1410'
      - '0x1438'
      - '0x143a'
      - '0x1fffff'
  filter:
    SourceImage|endswith:
      - '\werfault.exe'
      - '\MsMpEng.exe'
  condition: selection and not filter
level: high
tags:
  - attack.credential_access
  - attack.t1003.001
```

7.3 Sigma Rule — Office Application Spawning a Command/Script Interpreter

```yaml
title: Office Application Spawning Command or Script Interpreter
id: c2d3e4f5-0000-4a1b-9c3d-example000011
logsource:
  category: process_creation
  product: windows
detection:
  selection:
    ParentImage|endswith:
      - '\winword.exe'
      - '\excel.exe'
      - '\powerpnt.exe'
      - '\outlook.exe'
    Image|endswith:
      - '\powershell.exe'
      - '\cmd.exe'
      - '\wscript.exe'
      - '\cscript.exe'
      - '\mshta.exe'
  condition: selection
level: high
tags:
  - attack.execution
  - attack.t1059
  - attack.t1204.002
```

7.4 Splunk SPL — Kerberoasting Detection

```
index=wineventlog EventCode=4769 Ticket_Encryption_Type=0x17
| stats count dc(Service_Name) as unique_spns by Account_Name, src_ip
| where unique_spns > 5
```

7.5 Splunk SPL — New Service Installation Baseline Deviation

```
index=wineventlog EventCode=7045
| lookup approved_service_binaries.csv binary_path AS Service_File_Name OUTPUT approved
| where isnull(approved)
| table _time, ComputerName, Service_Name, Service_File_Name, Account_Name
```

7.6 KQL (Microsoft Sentinel / Defender) — Suspicious PowerShell Download Cradle

```
DeviceProcessEvents
| where FileName =~ "powershell.exe"
| where ProcessCommandLine has_any ("DownloadString", "IEX", "Invoke-Expression",
 ↪  "-EncodedCommand")
| project Timestamp, DeviceName, AccountName, ProcessCommandLine,
 ↪  InitiatingProcessFileName
```

7.7 KQL — Kerberoasting via Identity/Sign-in Logs (Hybrid AD Environments)

```
SecurityEvent
| where EventID == 4769
| where TicketEncryptionType == "0x17"
| summarize UniqueSPNs = dcount(ServiceName) by TargetUserName, IpAddress,
 ↪  bin(TimeGenerated, 5m)
| where UniqueSPNs > 5
```

7.8 Sysmon-Based Detection — Process Injection (CreateRemoteThread)

```yaml
title: Suspicious CreateRemoteThread into a Browser or LSASS
id: d3e4f5a6-0000-4a1b-9c3d-example000012
logsource:
  category: create_remote_thread
  product: windows
detection:
  selection:
    TargetImage|endswith:
      - '\lsass.exe'
      - '\chrome.exe'
      - '\firefox.exe'
  condition: selection
level: high
tags:
  - attack.defense_evasion
  - attack.t1055
```

---

8. Threat Hunting

8.1 Hunt: Suspicious Parent-Child Process Relationships

```
DeviceProcessEvents
| where InitiatingProcessFileName in~ ("winword.exe","excel.exe","outlook.exe")
| where FileName in~ ("powershell.exe","cmd.exe","wscript.exe","mshta.exe")
```

8.2 Hunt: LOLBin Abuse Patterns

```
DeviceProcessEvents
| where FileName =~ "rundll32.exe"
| where ProcessCommandLine has_any ("http://","https://","javascript:")
```

8.3 Hunt: Kerberoasting (Full Sweep)

Covered fully in Sections 7.4/7.7 — extend by also hunting for the specific tool signatures (`Rubeus`, `GetUserSPNs.py`) via Sysmon Event ID 1 command-line matching where EDR/Sysmon telemetry is available.

8.4 Hunt: Persistence Mechanism Sweep (Registry, Services, Tasks)

```
# Run Autoruns in command-line/report mode across the fleet, diff against a known-good
↪  baseline
autorunsc.exe -a * -c -h -s > autoruns_current.csv
```

8.5 Hunt: Living off the Land Binary Abuse

Maintain a curated watchlist of LOLBins (per the community-maintained LOLBAS project categories) and hunt for unusual argument patterns — e.g., `certutil.exe` with `-urlcache`/`-decode` flags (used to download/decode payloads disguised as certificate operations), or `mshta.exe` invoking a remote `.hta` URL.

8.6 Hunt: Anomalous WinRM/PowerShell Remoting Activity

```
DeviceNetworkEvents
| where RemotePort in (5985, 5986)
| where InitiatingProcessFileName != "svchost.exe"
```

8.7 Hunt: Data Staging and Exfiltration

```
DeviceFileEvents
| where FileName endswith ".zip" or FileName endswith ".rar"
| where FolderPath has_any (@"\AppData\Local\Temp", @"\Users\Public")
```

---

9. Incident Response

9.1 Containment

- Isolate the host at the network layer (EDR isolation, VLAN quarantine, or NIC disable), preserving running state for volatile evidence collection before any reboot.
- For confirmed credential compromise (Section 6.7), assume domain-wide impact if a privileged account was involved, and consider a broader precautionary credential rotation.

9.2 Eradication

- Remove all confirmed persistence mechanisms (Registry Run keys, malicious services, scheduled tasks, WMI subscriptions) found during the full sweep (Section 8.4).
- Rotate all credentials the compromised host/account had access to, including any cached domain credentials and, for LSASS-access incidents, treat every credential ever cached on that host as compromised.

9.3 Recovery

- Re-image from a known-good state where deep persistence or rootkit-level compromise is suspected — as with Linux (Chapter 3), the trustworthiness of a compromised host's own tools is fundamentally limited once significant privilege is confirmed lost.

9.4 Lessons Learned

- Any technique that evaded existing detection drives new Sigma/Sysmon content (Section 7) and updates to the baseline/hunt library (Section 8).

9.5 Evidence Collection (Order of Volatility)

1. **Live memory** (via a memory acquisition tool, e.g., Magnet RAM Capture, WinPMEM, or a cloud provider's snapshot capability)
2. **Network state**: `Get-NetTCPConnection`, active sessions
3. **Running process state**: full process listing with command lines, loaded modules, open handles
4. **Registry (live)**: relevant persistence-location snapshots
5. **Disk**: full disk image, or targeted collection of high-value artifacts (Section 10) if full imaging is infeasible
6. **Event Logs**: export all relevant `.evtx` files before any rotation/clearing can occur

```
# Example rapid evidence capture (illustrative)
$dest = "C:\Evidence\$(hostname)_$(Get-Date -Format yyyyMMdd_HHmmss)"
New-Item -ItemType Directory -Path $dest -Force
Get-Process | Export-Csv "$dest\processes.csv"
Get-NetTCPConnection | Export-Csv "$dest\connections.csv"
wevtutil epl Security "$dest\security.evtx"
wevtutil epl "Microsoft-Windows-Sysmon/Operational" "$dest\sysmon.evtx"
```

9.6 Timeline Creation

Combine Event Log timestamps, Sysmon process creation times, file MACB timestamps (Section 2.6), and DFIR artifact timestamps (Section 10) into a single UTC-normalized chronological view — tools like **Timeline Explorer** (Eric Zimmerman's tools) or a well-structured spreadsheet/SIEM query serve this purpose in practice.

9.7 Chain of Custody

Hash every collected `.evtx` export and disk image (SHA-256) immediately, record collector identity/timestamp/tool version, and store evidence in an access-controlled, tamper-evident location separate from production.

---

10. Digital Forensics — Windows DFIR Artifacts Deep Dive

Artifact	Location	Evidence Provided
Event Logs	`C:\Windows\System32\winevt\Logs\*.evtx`	The primary structured record of security-relevant OS activity (Section 2.14-2.15)
Registry	`C:\Windows\System32\config\` (SYSTEM, SOFTWARE, SAM, SECURITY hives), `NTUSER.DAT` per user	Persistence, configuration, and (via `RegIdleBackup`/volume shadow copies) historical state
Prefetch	`C:\Windows\Prefetch\*.pf`	Proves a specific executable ran, including run count and last-run (and, in newer versions, up to 8 previous run) timestamps — valuable even after the original binary is deleted
Amcache	`C:\Windows\AppCompat\Programs\Amcache.hve`	Records executed binaries including full file path, SHA-1 hash, and first-execution timestamp — extremely valuable for proving exactly what ran and when, even for since-deleted files

Artifact	Location	Evidence Provided
ShimCache (AppCompatCache)	Registry: `SYSTEM\CurrentControlSet\Control\Session Manager\AppCompatCache`	Records file metadata (path, size, last modified) for executed or merely *examined* binaries, providing execution-adjacent evidence, though with important nuance around what "presence" actually proves
BAM/DAM (Background/Desktop Activity Moderator)	Registry: `SYSTEM\CurrentControlSet\Services\bam\...`	Records the last execution time of an application per user SID, a valuable additional execution-evidence source on modern Windows versions
SRUM (System Resource Usage Monitor)	`C:\Windows\System32\sru\SRUDB.dat`	Historical per-application network usage and resource consumption, useful for retrospective data-volume/exfiltration estimation even after the process is long gone
LNK Files	`C:\Users\<user>\AppData\Roaming\Microsoft\Windows\Recent\`	Shortcut files automatically created when a user opens a document/application, recording the original file's path, timestamps, and the volume it was opened from — valuable for proving a specific file was accessed, even from removable media no longer present
Jump Lists	`C:\Users\<user>\AppData\Roaming\Microsoft\Windows\Recent\AutomaticDestinations\` / `CustomDestinations\`	Per-application recent-file history, similar evidentiary value to LNK files but application-specific
Browser History	Browser-specific profile directories (e.g., Chrome's `History` SQLite database)	Web activity, useful for phishing-click reconstruction and insider-threat/data-staging investigations

Artifact	Location	Evidence Provided
MFT	`$MFT` on each NTFS volume	Complete file/directory metadata for the volume — the single richest file-system-level forensic structure (Section 2.6)
USN Journal	`$UsnJrnl` on each NTFS volume	Change history log, valuable for reconstructing file activity even after deletion (Section 2.6)
Recycle Bin	`$Recycle.Bin\<user SID>\`	Deleted file recovery, retains original path and deletion timestamp metadata
PowerShell History	`C:\Users\<user>\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\ConsoleHost_history.txt`	Commands typed in PowerShell sessions, analogous to Linux bash history (Chapter 3, Section 4.1) and equally subject to tampering/clearing
Windows Defender Logs	`C:\ProgramData\Microsoft\Windows Defender\Support\` and Event Log (`Microsoft-Windows-Windows Defender/Operational`)	Detection history, scan results, and (importantly) any detections that were later excluded/allowed, which can itself be a significant investigative lead

```
# Query Prefetch files for a specific executable's run history
Get-ChildItem 'C:\Windows\Prefetch\*.pf' | Where-Object { $_.Name -match 'SUSPECT' }

# Parse Amcache (requires a dedicated parser like AmcacheParser from Eric Zimmerman's
 ↪  tools in practice)
# Illustrative — direct registry hive parsing requires offline/dedicated tooling

# Query BAM data
Get-ItemProperty -Path
 ↪  'HKLM:\SYSTEM\CurrentControlSet\Services\bam\State\UserSettings\*'
```

Why using multiple artifact sources together matters: No single artifact is fully tamper-proof or complete on its own — Prefetch can be disabled/cleared, Event Logs can be cleared (Event ID 1102), and ShimCache/Amcache each have specific nuances in what they actually prove. A confident DFIR conclusion is built by **triangulating across several independent sources** (e.g., confirming a binary ran using Prefetch + Amcache + Sysmon Event ID 1 + the corresponding network connection in Sysmon Event ID 3), exactly mirroring the multi-source corroboration principle emphasized throughout this book (Chapters 1-5).

---

11. Common Malware Techniques — Windows Persistence and Attack Techniques

11.1 Persistence Techniques (Normal → Attack → Artifact → Detection)

11.1.1 Registry Run Keys / RunOnce

- **Normal:** Legitimate software (updaters, tray utilities) registers itself to launch at logon via `HKLM`/`HKCU ...\Run`.
- **Attack:** Malware adds its own entry, executing on every subsequent logon.
- **Artifact:** New value under `Run`/`RunOnce` keys.
- **Detection:** Sysmon Event ID 13 (Registry value set) on these specific paths; Autoruns baseline diffing.
- **MITRE ATT&CK:** T1547.001.

11.1.2 Startup Folder

- **Normal:** Shortcuts placed by installers for legitimate auto-launch applications.
- **Attack:** A malicious executable or shortcut is placed in the per-user or all-users Startup folder.
- **Artifact:** New file in `...\Start Menu\Programs\Startup`.
- **Detection:** Sysmon Event ID 11 (File Create) scoped to Startup folder paths.
- **MITRE ATT&CK:** T1547.001.

11.1.3 Services

- Covered in depth in Section 2.11 — **MITRE ATT&CK:** T1543.003.

11.1.4 Scheduled Tasks

- Covered in depth in Section 2.12 — **MITRE ATT&CK:** T1053.005.

11.1.5 WMI Event Subscriptions

- **Normal:** WMI (Windows Management Instrumentation) legitimately supports event-driven automation for systems management.
- **Attack:** An attacker registers a permanent WMI event subscription (an `__EventFilter` bound via `__FilterToConsumerBinding` to an `__EventConsumer`) that executes a payload in response to a system event (e.g., every logon, or on a timer) — a stealthy persistence mechanism since it lives in the WMI repository rather than the Registry or a visible file.
- **Artifact:** Entries in the WMI repository (`root\subscription` namespace).
- **Detection:** Sysmon Event IDs 19 (WmiEventFilter), 20 (WmiEventConsumer), 21 (WmiEventConsumerToFilter) — specifically designed to surface this otherwise hard-to-see persistence mechanism.
- **MITRE ATT&CK:** T1546.003 (Windows Management Instrumentation Event Subscription).

11.1.6 DLL Search Order Hijacking

- **Normal:** Windows searches a defined sequence of directories to locate a DLL an application requests by name (rather than full path).
- **Attack:** An attacker places a malicious DLL with the same name as a legitimate one in a directory earlier in the search order (often the application's own directory), causing the application to load the attacker's DLL instead.
- **Artifact:** An unexpected DLL in an application directory, or Sysmon Event ID 7 (Image Loaded) showing a DLL loaded from an unusual path.
- **Detection:** Sysmon 7 with path-based anomaly rules; comparing loaded DLL paths against expected installation directories.
- **MITRE ATT&CK:** T1574.001.

11.1.7 Winlogon Hijacking

- Covered in Section 2.9 (`Shell`/`Userinit` values) — **MITRE ATT&CK:** T1547.004 (Winlogon Helper DLL).

11.1.8 AppInit DLLs

- Covered in Section 2.9 — **MITRE ATT&CK:** T1546.010.

11.1.9 Browser Extension Persistence

- **Normal:** Users/administrators legitimately install browser extensions for productivity/security tooling.
- **Attack:** A malicious browser extension (sideloaded or installed via a compromised profile) provides persistent access to browsing sessions, credentials, and can serve as a C2/data-exfiltration channel that blends into normal browser network traffic.
- **Artifact:** Unexpected extension entries in the browser's profile configuration.
- **Detection:** Periodic extension inventory auditing against an approved list; unusual outbound connections originating from the browser process correlating with extension activity.
- **MITRE ATT&CK:** T1176 (Browser Extensions).

11.1.10 COM Hijacking

- **Normal:** COM objects are legitimately registered under `HKCR\CLSID\{GUID}\InprocServer32`, pointing to the DLL implementing that object's functionality.
- **Attack:** An attacker modifies (or, for a not-yet-registered CLSID, creates) this registry path to point to a malicious DLL, which then loads whenever any application instantiates that COM object.
- **Artifact:** Modified `InprocServer32` value pointing to an unexpected path.
- **Detection:** Sysmon Event ID 13 (Registry value set) on CLSID paths; comparing against a known-good COM registration baseline.
- **MITRE ATT&CK:** T1546.015 (Component Object Model Hijacking).

11.2 Attack Techniques — Defensive Overview

Technique	Defensive Summary	Key Detection Signal	MITRE ATT&CK
Brute Force	Repeated logon attempts until success	4625/4771 volume from one source	T1110
Password Spraying	Few attempts across many accounts to avoid lockout	Many distinct 4625/4771 target accounts, one source	T1110.003
Credential Dumping	Extracting credential material from memory/disk (LSASS, SAM, NTDS.dit)	Sysmon 10 LSASS access; NTDS.dit access outside backup windows	T1003
LSASS Abuse	Direct memory reading of the LSASS process for plaintext/hash/ticket extraction	Sysmon 10 with suspicious GrantedAccess values	T1003.001
Pass-the-Hash	Authenticating using a stolen NTLM hash without knowing the plaintext password	4624 Logon Type 3/9 anomalies with NTLM in unexpected contexts	T1550.002
Pass-the-Ticket	Authenticating using a stolen/forged Kerberos ticket	Anomalous 4624/4769 patterns, ticket reuse across hosts	T1550.003
Kerberoasting	Requesting and offline-cracking service tickets	4769 RC4 burst across many SPNs	T1558.003
AS-REP Roasting	Targeting accounts without Kerberos pre-authentication	4768 pattern for pre-auth-disabled accounts	T1558.004
UAC Bypass	Silently obtaining an elevated token without a consent prompt	Sysmon 1 showing known bypass-technique process chains (e.g., abuse of specific auto-elevating binaries)	T1548.002
DLL Hijacking	Search-order or side-loading DLL substitution	Sysmon 7 path anomalies	T1574.001/.002

Technique	Defensive Summary	Key Detection Signal	MITRE ATT&CK
Process Injection	Writing/executing code within another process's memory space	Sysmon 8 (CreateRemoteThread), Sysmon 10	T1055
Living off the Land / LOLBins	Using pre-installed legitimate binaries for malicious purposes	Unusual argument patterns on common binaries (Section 8.5)	T1218 and related
PowerShell Abuse	Encoded/obfuscated/in-memory script execution	Event ID 4104, Sysmon 1	T1059.001
WMI Abuse	Remote execution or persistence via WMI	Sysmon 19/20/21, `wmic.exe`/`wmiprvse.exe` anomalies	T1047, T1546.003
Remote Services (RDP/SMB/WinRM)	Lateral movement using legitimate remote administration channels	4624 Logon Type 3/10, corresponding network traffic	T1021.001/.002/.006
PsExec	Legitimate remote execution tool frequently abused for lateral movement	Temporary service creation (7045) with a PsExec-pattern name, named pipe artifacts	T1569.002
Windows Admin Shares	`C$`/`ADMIN$` used for remote file transfer/execution	Sysmon 3/SMB traffic to admin shares from non-management hosts	T1021.002

---

12. Blue Team Perspective

Mature Windows-focused blue teams treat Sysmon deployment (with a well-tuned, community-vetted configuration) and PowerShell Script Block Logging as non-negotiable baseline telemetry, since both close visibility gaps the built-in Security log leaves wide open. Detection engineering follows the same MITRE ATT&CK-driven workflow established in prior chapters: identify a technique, determine the best available log source (Sysmon nearly always outperforms the Security log alone for process/network/registry-level detail), write and tune a Sigma/KQL/SPL rule, and validate against both legitimate administrative activity (to control false positives) and, where possible, a red-team-executed sample of the actual technique. Fleet-wide baseline diffing (Autoruns sweeps, new-service monitoring, Registry persistence-key monitoring) provides a scalable complement to individual alert-driven investigation, directly mirroring the Linux baseline-diffing approach from Chapter 3.

13. Red Team Perspective

Red teams targeting Windows environments favor **Living off the Land** techniques for the same reason as on Linux (Chapter 3) — using PowerShell, WMI, and built-in LOLBins avoids dropping new, potentially signature-detectable binaries. Credential-focused techniques (LSASS dumping, Kerberoasting, Pass-the-Hash/Ticket) are disproportionately valuable in Windows/Active-Directory environments specifically because successful credential theft often grants immediate, broad lateral movement capability across an entire domain — a difference in blast radius from most Linux privilege escalation, which more often stays host-scoped absent shared credential reuse. Mature red teams specifically test their own tooling against Sysmon/EDR telemetry before an engagement, anticipating exactly the detection techniques covered in Section 7.

---

14. Real Case Studies

14.1 Case Study: Full Attack Chain — Phishing → Execution → Persistence → Credential Access → Lateral Movement → C2 → Exfiltration

Background: A composite, realistic scenario reflecting patterns consistently documented across public incident response reports involving Windows/Active-Directory environments — directly comparable in structure to numerous real-world ransomware and targeted-intrusion write-ups from major IR/MDR vendors.

1. Phishing: A spear-phishing email delivers a macro-enabled Word document to a target employee.

2. Execution: The macro spawns `powershell.exe` with an encoded, obfuscated command that downloads and executes a further-stage payload entirely in memory (Section 2.16) — a pattern directly detectable via Section 7.3's Sigma rule (Office app spawning a script interpreter).

3. Persistence: The payload establishes persistence via a Registry Run key and a WMI permanent event subscription for redundancy (Sections 11.1.1/11.1.5).

4. Credential Access: The attacker uses a credential-dumping tool to read LSASS memory (Section 6.7), harvesting cached credentials including a privileged domain account that had recently logged onto the compromised workstation.

5. Lateral Movement: Using the harvested privileged credentials, the attacker moves laterally via WinRM/PowerShell Remoting and SMB admin shares to several servers, including a domain controller.

6. C2: A more capable C2 implant is deployed on the domain controller, beaconing out over HTTPS disguised as normal web traffic.

7. Exfiltration: Sensitive data is archived and exfiltrated over the existing C2 channel or a separate cloud-storage upload.

Detection Opportunities at Each Stage:

Stage	Detection Opportunity
Execution	Sigma rule for Office-spawned interpreters (7.3); Event ID 4104 script content
Persistence	Sysmon 13 (Registry) and Sysmon 19-21 (WMI subscription)
Credential Access	Sysmon 10 LSASS access (7.2)
Lateral Movement	4624 Logon Type 3 anomalies, WinRM/SMB traffic to unexpected hosts
C2	Beaconing/protocol-mismatch detection (Chapters 1-4 techniques)
Exfiltration	Archive-creation-before-transfer and large-upload detection (8.7)

Lessons Learned: As with the Linux equivalent chain (Chapter 3, Section 14.1), no single control would have stopped this intrusion alone — layered detection across execution, persistence, credential access, and network stages is what ultimately provides resilient coverage, and the *credential access* stage is disproportionately high-value to prevent/detect early in Windows/AD environments given how directly it enables broad lateral movement.

---

15. Common Mistakes

Mistake	Why It Happens	How to Avoid It
Relying solely on the Security log without Sysmon	Assuming built-in logging is sufficient	Deploy Sysmon fleet-wide; it captures network, DLL-load, and registry detail the Security log does not
Not enabling command-line auditing for 4688	Overlooking the non-default configuration requirement	Explicitly enable command-line inclusion (Section 3.2) as a baseline hardening step
Treating a raw PowerShell command line as the full picture	Not accounting for encoding/obfuscation	Always pull Event ID 4104 for de-obfuscated content
Checking only one persistence location	Not realizing how many distinct Windows persistence mechanisms exist	Run a full Autoruns-based sweep (Section 8.4) covering Registry, services, tasks, and WMI together

Mistake	Why It Happens	How to Avoid It
Assuming UAC prevents privilege escalation	Misunderstanding UAC's design intent	Treat UAC as a consent/convenience feature, not a security boundary; rely on least-privilege and monitoring instead
Ignoring ShimCache/ Amcache/Prefetch nuances	Treating any of them as unconditionally proving execution	Understand each artifact's specific evidentiary meaning and corroborate across multiple sources (Section 10)
Not rotating credentials broadly enough after an LSASS-dumping incident	Underestimating blast radius	Treat every credential ever cached on the affected host as compromised, not just the account under direct investigation

---

16. Interview Questions

An extensive, representative interview question bank covering Windows technical fundamentals, SOC scenarios, DFIR, and senior-level questions, each with a complete answer and reasoning.

16.1 Windows Technical Questions

Q1: What's the difference between user mode and kernel mode? *A:* Kernel mode has full, unrestricted hardware/memory access (the NT kernel, HAL, drivers); user mode is restricted, isolated per-process memory where applications and most services run. Transitions between them happen via a controlled system call interface, and kernel-mode compromise (e.g., a vulnerable driver) grants total system control, making it a high-value privilege escalation target.

Q2: What is a security token, and why does it matter for attackers? *A:* A token represents a process/thread's security context — SID, group memberships, privileges. It matters because token theft/impersonation lets an attacker act as a higher-privileged user without needing their password, and specific privileges (like `SeImpersonatePrivilege`) are documented privilege-escalation vectors.

Q3: Explain the difference between the MFT and the USN Journal. *A:* The MFT is NTFS's central metadata table holding a record (including timestamps and, for small files, content) for every file/directory on the volume — essentially the current-state snapshot. The USN Journal is a continuously-updated change log recording every create/modify/rename/delete event, useful for reconstructing activity history, including for files that have since been deleted from the MFT's current view.

Q4: What are Alternate Data Streams and why are they a security concern? *A:* NTFS allows a file to have multiple named data streams beyond its primary content. They're a concern because malware has

historically hidden payloads in a secondary stream, invisible in standard directory listings, effectively hiding data “inside” an innocuous-looking file.

Q5: What’s the difference between HKLM and HKCU? *A:* HKLM (HKEY_LOCAL_MACHINE) holds system-wide configuration affecting all users; HKCU (HKEY_CURRENT_USER) holds configuration specific to the currently logged-on user (a view into that user’s subkey under HKU).

Q6: Why is `svchost.exe` both extremely common and a common malware masquerade target? *A:* It’s the generic host process for services implemented as DLLs rather than standalone EXEs, so dozens of legitimate instances run simultaneously — this ubiquity makes it an appealing name for malware to masquerade as, hoping to blend in; the key differentiator is that legitimate `svchost.exe` always has `services.exe` as its parent and a `-k` command-line argument specifying its service group.

Q7: What does Event ID 4688 show, and what critical configuration does it require to be useful? *A:* It logs process creation. By default the command line isn’t included — the `ProcessCreationIncludeCmdLine_Enabled` registry setting (or equivalent Group Policy) must be explicitly enabled, without which 4688 shows a process ran but not with what arguments, dramatically limiting its investigative value.

Q8: What is the difference between Sysmon Event ID 1 and the Security log’s Event ID 4688? *A:* Both log process creation, but Sysmon 1 includes richer detail by default (full command line without extra configuration, parent image, hashes, and integrates with the same tool’s network/registry/file events), whereas 4688 requires additional configuration for command-line visibility and is part of the built-in OS rather than a separately-installed tool.

Q9: Explain Kerberoasting and why RC4 encryption is a specific red flag. *A:* Kerberoasting requests service tickets (TGS-REP) for accounts with an SPN, then attempts offline cracking since the ticket is encrypted with the service account’s password hash. RC4 (etype 0x17) is a red flag because it’s weaker and faster to crack offline than AES, and modern legitimate Kerberos traffic in a well-configured environment should predominantly use AES — a burst of RC4-encrypted TGS requests for many distinct SPNs from one account is a strong Kerberoasting indicator.

Q10: What is Script Block Logging and why is it more valuable than the raw PowerShell command line? *A:* Script Block Logging (Event ID 4104) records the actual, de-obfuscated content PowerShell executes, regardless of how heavily the original invocation was encoded/obfuscated — since PowerShell must resolve code to its real form before execution, this closes the gap that raw command-line logging alone leaves open against obfuscation.

Q11-Q50: *(An extensive additional technical question set continues in the same style, covering: NTLM vs. Kerberos authentication flow differences; the purpose and risk of AppInit_DLLs; COM hijacking mechanics; the difference between Prefetch and Amcache evidentiary value; WMI event subscription structure; DLL search order and hijacking mechanics; UAC’s split-token model and why it isn’t a security boundary; the difference between local and domain accounts; Windows Firewall rule evaluation order; the purpose of the SAM database versus NTDS.dit; Constrained Language Mode’s protective scope and limits; the difference between Pass-the-Hash and Pass-the-Ticket; WinRM’s role in lateral movement; the significance of Logon Type values (2, 3, 4, 5, 7, 9, 10, 11); the purpose and abuse potential of*

`regsvr32.exe`/`mshta.exe`/`rundll32.exe`; Group Policy's security-relevant capabilities and risks if compromised; the difference between a Domain Admin and Enterprise Admin; the purpose of Protected Users group and Credential Guard; and the general architecture of Active Directory replication and its relevance to detecting NTDS.dit extraction attempts.)

16.2 Windows SOC Scenario Questions

Q51: A workstation shows `winword.exe` spawning `powershell.exe` with an encoded command. Walk through your investigation. *A:* Decode the command (Base64/UTF-16LE) to reveal its true content; check Event ID 4104 for the full de-obfuscated script if Script Block Logging is enabled; trace any resulting network connections (Sysmon 3) or dropped files (Sysmon 11); identify the phishing email/document as the likely root cause and check for other recipients of the same campaign.

Q52: You observe Sysmon Event ID 10 showing `notepad.exe` accessing `lsass.exe` with a high-privilege access mask. What's your assessment? *A:* Highly suspicious — `notepad.exe` has no legitimate reason to access LSASS memory with that access level; this pattern (a renamed/masquerading legitimate-looking binary performing LSASS access) is consistent with credential-dumping tooling disguising its process name, and should be treated as a high-severity finding requiring immediate host isolation and credential rotation.

Q53-Q80: *(Additional scenario questions covering: a new Domain Admin group member added outside a change window; a burst of 4769 events with RC4 encryption; a service created with a base64-encoded PowerShell binary path; WinRM connections from a finance workstation to a domain controller; a cleared Security log (Event ID 1102) discovered during routine review; an AppInit_DLLs registry value pointing to an unsigned DLL in a temp directory; lateral movement via PsExec-pattern service creation across multiple hosts within minutes; and a suspicious LNK file referencing a no-longer-present USB drive found in a user's Recent folder during an insider-threat investigation — each worked through using the ten-workflow framework from Section 6.)*

16.3 Windows DFIR Questions

Q81: Why might Prefetch alone be insufficient to prove when a file first executed? *A:* Prefetch retains a limited history of run timestamps (historically the last run time, with newer Windows versions retaining several previous run timestamps) and can be disabled or cleared; corroborating with Amcache (which records a first-execution timestamp) and Sysmon Event ID 1 (if deployed from before the activity) provides a more complete and resilient picture.

Q82: What's the evidentiary nuance around ShimCache (AppCompatCache) that analysts often get wrong? *A:* ShimCache entries do not definitively prove a file executed — under some circumstances, simply being examined by the OS's compatibility-checking logic can create an entry without execution occurring. Analysts should treat ShimCache as corroborating, not conclusive, evidence, and cross-reference with Amcache/Prefetch/Sysmon for a confident execution determination.

Q83-Q100: *(Additional DFIR questions covering: the difference in evidentiary value between BAM and Prefetch; how Jump Lists can prove file access from removable media no longer available for imaging;*

SRUM's value for retrospective data-usage estimation in exfiltration investigations; the correct order-of-volatility evidence collection sequence; how the Recycle Bin retains metadata after "permanent" deletion via Shift+Delete in some configurations; and how to build a unified Windows timeline from Event Logs, Sysmon, and filesystem metadata.)

16.4 Senior-Level Questions

Q101: How would you design a Windows fleet-wide persistence detection strategy at enterprise scale? *A:* Deploy Sysmon fleet-wide with a tuned configuration; centralize Event Log forwarding (WEF or SIEM agents) so local log clearing (Event ID 1102) doesn't destroy the only copy of evidence; run scheduled Autoruns-based sweeps (or equivalent EDR persistence-inventory features) diffed against a maintained known-good baseline; and prioritize detection content for the highest-value single events (7045 new service, 4698 new task, Sysmon 13 on Run keys/Winlogon/AppInit paths, Sysmon 19-21 for WMI subscriptions) since these cover the overwhelming majority of real-world persistence techniques from a manageable number of monitored events.

Q102: How do you balance PowerShell Script Block Logging's log volume against its detection value at scale? *A:* Script Block Logging can generate significant volume, particularly from legitimate automation/configuration-management scripts; mitigate by ensuring SIEM ingestion/storage is sized appropriately, applying targeted exclusions for known-legitimate high-volume automation sources (documented and reviewed periodically, not blanket-excluded), and prioritizing alerting logic (rather than log volume reduction) on high-signal patterns (`DownloadString`, `-EncodedCommand`, `Invoke-Expression`) rather than attempting to suppress the underlying logging itself.

Q103-Q120: *(Additional senior-level questions covering: designing a credential-hygiene program to limit Pass-the-Hash/Ticket blast radius (tiered administration model, Protected Users group, Credential Guard); building a detection engineering pipeline that validates new Sigma content against both legitimate and red-team-generated traffic before production deployment; articulating Active Directory hardening priorities to executive leadership; and designing a DFIR evidence-retention policy that balances storage cost against the realistic dwell-time distribution seen in the organization's own incident history.)*

---

17. Real Interview Scenarios

Scenario A: "The Quiet WMI Persistence"

Alert: A threat hunt sweep (Section 8.4) using Sysmon Event IDs 19-21 reveals a WMI permanent event subscription with no corresponding change management record, bound to a `__IntervalTimerInstruction` filter firing every 60 minutes.

Investigation: The hunter extracts the consumer's action, revealing an encoded PowerShell command matching the download-cradle pattern from Section 7.6. Cross-referencing Sysmon Event ID 1 for the timestamp of the subscription's registration reveals it was created moments after a Sysmon 10 LSASS-access event on the same host two weeks earlier — indicating the WMI persistence was established as a secondary/redundant access mechanism following an initial credential-dumping compromise that had otherwise gone undetected

at the time.

Root Cause: Initial compromise via credential dumping was not caught in real time (no alerting was configured on the Sysmon 10 LSASS-access pattern at the time), allowing the attacker to establish durable, low-visibility WMI-based persistence well before detection.

MITRE ATT&CK: T1003.001 (initial credential access), T1546.003 (WMI persistence).

Final Recommendation: Implement real-time alerting on Sysmon Event ID 10 LSASS access patterns (Section 7.2) rather than relying solely on retrospective hunting; extend the persistence baseline sweep to include WMI subscriptions on a recurring, automated schedule rather than only during incident-driven investigations.

Scenario B: "The Service That Wasn't in Change Management"

Alert: Fleet-wide Event ID 7045 monitoring (Section 5.4) flags a new service on a file server with a binary path pointing to `C:\Windows\Temp\svchost32.exe` — a name deliberately chosen to resemble the legitimate `svchost.exe`.

Investigation: The analyst confirms the binary's location (`Temp`, not `System32`) and name are both immediately anomalous. Sysmon Event ID 1 reveals the service was created by a process chain originating from an RDP session (Logon Type 10) from an internal workstation, itself preceded by a burst of 4625 failures against the file server's local Administrator account minutes earlier — indicating a successful RDP brute force followed by service-based persistence establishment.

Root Cause: The file server had RDP exposed internally with a weak/guessable local Administrator password and no account lockout policy enforced, permitting a successful brute force.

MITRE ATT&CK: T1110 (Brute Force), T1021.001 (RDP), T1543.003 (Windows Service persistence).

Final Recommendation: Enforce account lockout policy and strong password requirements for all local administrator accounts; disable or tightly restrict internal RDP access to only jump-host-mediated connections; extend the 7045 baseline-deviation alert (Section 5.4) to a standing, always-on detection rather than a manually-triggered sweep.

---

18. Hands-on Labs

Lab 1 (Beginner): Event Log and Process Fundamentals

Objective: Practice core Windows Event Log querying and process investigation commands.

Environment: Any Windows VM (lab/test environment).

Scenario: A user reports their account was locked out; determine why.

Commands:

```
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4625,4740} -MaxEvents 50 |
  Select-Object TimeCreated, Id, Message
```

Questions: How many failed logon attempts preceded the lockout? From what source? Was the account subsequently used successfully from a different source?

Expected Results: A clear count and source attribution for the failed attempts leading to the 4740 lockout event.

Solution: Confirm findings by cross-referencing the source IP/workstation against the account owner's expected normal location.

MITRE Mapping: T1110.

Lab 2 (Intermediate): Registry Persistence Discovery

Objective: Find and analyze a planted Registry persistence mechanism.

Environment: A lab VM with an instructor-planted (or self-planted, for self-study) Run key entry.

Scenario: A host is suspected of having malware persistence; no obviously malicious running process is currently visible.

Commands:

```
Get-ItemProperty -Path 'HKLM:\Software\Microsoft\Windows\CurrentVersion\Run'
Get-ItemProperty -Path 'HKCU:\Software\Microsoft\Windows\CurrentVersion\Run'
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational'; Id=13}
 ↪  -MaxEvents 100
```

Logs: Correlate the Sysmon Event ID 13 timestamp for the Run key's creation against Sysmon Event ID 1 for the process responsible.

Questions: Which Run key location contained the malicious entry? What does it execute? What process created it, and when?

Expected Results: Full identification of the planted entry, its payload, and the responsible process chain.

Solution: The entry should be traced back to a specific process (e.g., a downloaded and executed dropper) via the Sysmon 13-to-Sysmon 1 correlation.

MITRE Mapping: T1547.001.

Lab 3 (Advanced): Full Phishing-to-Lateral-Movement Chain Investigation

Objective: Investigate a simulated multi-stage compromise end-to-end.

Environment: An isolated, authorized lab Active Directory environment with a planted phishing-macro execution chain, Registry/WMI persistence, an LSASS-access simulation, and a lateral movement path to a

second host (instructor-prepared, or self-built for self-study using the techniques described throughout this chapter).

Scenario: A workstation shows anomalous PowerShell activity; management wants a full root-cause investigation and incident report covering the entire chain through to any lateral movement.

Commands:

```
# Process/PowerShell triage
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-PowerShell/Operational';
 ↪  Id=4104} -MaxEvents 50
Get-WinEvent -FilterHashtable @{LogName='Microsoft-Windows-Sysmon/Operational';
 ↪  Id=1,3,10,13} -MaxEvents 200

# Persistence sweep
autorunsc.exe -a * -c -h -s

# Lateral movement check
Get-WinEvent -FilterHashtable @{LogName='Security'; Id=4624} -MaxEvents 100 |
 ↪  Where-Object {$_.Message -match 'Logon Type:\s+3'}
```

Logs: Full correlation across Sysmon (process/network/registry), Security log (authentication), and PowerShell Operational (script content) logs.

Questions: What was the initial access vector? What persistence mechanisms were established? Was credential access achieved, and did lateral movement follow? What was the full timeline?

Expected Results: A complete attack chain narrative matching the pattern in Section 14.1, with supporting evidence at every stage.

Solution: Full write-up following the Section 6 ten-workflow framework for each stage discovered, culminating in a unified timeline (Section 9.6) and MITRE ATT&CK mapping.

MITRE Mapping: T1566.001 (phishing), T1059.001 (PowerShell execution), T1547.001/T1546.003 (persistence), T1003.001 (credential access), T1021 (lateral movement).

---

19. Cheat Sheet

19.1 Important Paths

Path	Purpose
`C:\Windows\System32\config\`	Registry hive files (SAM, SECURITY, SYSTEM, SOFTWARE)
`C:\Windows\Prefetch\`	Execution evidence (.pf files)

Path	Purpose
`C:\Windows\AppCompat\Programs\Amcache.hve`	Execution evidence with hashes
`C:\Windows\System32\winevt\Logs\`	Event Log files (.evtx)
`C:\Windows\System32\sru\SRUDB.dat`	Resource usage history
`...\AppData\Roaming\Microsoft\Windows\Recent\`	LNK files / Jump Lists
`...\AppData\Roaming\Microsoft\Windows\PowerShell\PSReadLine\`	PowerShell command history
`$Recycle.Bin\<SID>\`	Deleted file recovery
`C:\Windows\System32\Tasks\`	Scheduled task XML definitions

19.2 Important Registry Keys

Key	Purpose
`HKLM/HKCU\...\CurrentVersion\Run` / `RunOnce`	Logon persistence
`HKLM\...\Winlogon` (`Shell`, `Userinit`)	Logon process hijacking
`HKLM\...\Windows` (`AppInit_DLLs`)	Process-wide DLL injection
`HKCR\CLSID\{GUID}\InprocServer32`	COM hijacking
`HKLM\System\CurrentControlSet\Services\<name>`	Service configuration
`HKLM\SYSTEM\...\AppCompatCache`	ShimCache
`HKLM\SYSTEM\...\Services\bam\State\UserSettings`	BAM execution evidence

19.3 Critical Event IDs

`4624`/`4625` Logon Success/Failure · `4634` Logoff · `4648` Explicit Credential Logon · `4672` Special Privileges · `4688` Process Creation · `4697`/`7045` Service Installed · `4698` Scheduled Task Created · `4720` Account Created · `4728`/`4732` Group Membership Added · `4740` Account Locked · `4768`/`4769`/`4771` Kerberos TGT/TGS/Pre-Auth-Failed · `1102` Audit Log Cleared

19.4 Sysmon Event IDs

`1` Process Creation · `2` File Creation Time Changed · `3` Network Connection · `5` Process Terminated · `6` Driver Loaded · `7` Image Loaded · `8` CreateRemoteThread · `10` Process Access · `11` File Create · `12/13/14` Registry Events · `19/20/21` WMI Events · `22` DNS Query

19.5 Important Processes

`svchost.exe` · `lsass.exe` · `winlogon.exe` · `services.exe` · `explorer.exe` · `rundll32.exe` · `regsvr32.exe` · `mshta.exe` · `powershell.exe` · `cmd.exe` · `wscript.exe`/`cscript.exe`

19.6 Network Ports

`88` Kerberos · `135` RPC · `139/445` SMB · `389/636` LDAP/LDAPS · `3389` RDP · `5985/5986` WinRM

19.7 DFIR Artifacts Quick Reference

Prefetch · Amcache · ShimCache · BAM/DAM · SRUM · LNK Files · Jump Lists · MFT · USN Journal · Recycle Bin · PowerShell History · Windows Defender Logs

19.8 MITRE ATT&CK Quick Reference (This Chapter)

`T1078` Valid Accounts · `T1110/.003` Brute Force/Spraying · `T1003.001` LSASS Dumping · `T1550.002/.003` Pass-the-Hash/Ticket · `T1558.003/.004` Kerberoasting/AS-REP Roasting · `T1059.001` PowerShell · `T1047` WMI · `T1053.005` Scheduled Task · `T1543.003` Windows Service · `T1547.001` Run Keys/Startup · `T1546.003` WMI Subscription · `T1546.010` AppInit DLLs · `T1546.015` COM Hijacking · `T1574.001` DLL Search Order Hijacking · `T1055` Process Injection · `T1021` Remote Services · `T1070.001` Clear Event Logs

---

20. Summary

20.1 Key Points

- Windows provides an extraordinarily rich native telemetry surface (Event Logs, Registry, Prefetch, Amcache, ShimCache), but interpreting it correctly requires deep, specific knowledge of what each source actually proves and its limitations.
- Sysmon and PowerShell Script Block Logging close critical visibility gaps the built-in Security log leaves open, and should be considered baseline, non-negotiable SOC tooling.
- The parent-child process relationship heuristic (“does this process’s parent make sense?”) is the single highest-value general detection principle across nearly every Windows attack technique.
- Credential access (LSASS dumping, Kerberoasting, Pass-the-Hash/Ticket) is disproportionately consequential in Windows/Active-Directory environments due to the broad lateral movement it typically enables.
- No single DFIR artifact is fully conclusive on its own — confident investigation requires triangulating across multiple independent sources (Prefetch + Amcache + Sysmon + network telemetry).
- Persistence spans many distinct locations (Registry, services, scheduled tasks, WMI, COM, AppInit_DLLs) requiring a full, disciplined sweep rather than checking any single location alone.

20.2 Revision Notes

Revisit: the full Event ID reference table (Section 2.15), the ten SOC investigation workflows (Section 6), the DFIR artifact evidentiary-value table (Section 10), and the Normal → Attack → Artifact → Detection structure applied throughout Section 11.

20.3 Most Important Interview Topics

Event ID meaning and investigation steps, Sysmon's advantages over the built-in Security log, PowerShell Script Block Logging, LSASS credential dumping detection, Kerberoasting/AS-REP Roasting mechanics, and the evidentiary nuances of Prefetch/Amcache/ShimCache.

20.4 Most Common Exam Questions

Explaining what a specific Event ID means and how to investigate it; describing the Kerberos authentication flow and its attack variants; explaining UAC's actual security model and limitations; identifying all major Windows persistence locations; and walking through a phishing-to-lateral-movement attack chain with correct MITRE ATT&CK mapping at each stage.

End of Chapter 4 - Chapter 5 Coming Soon

Prepared by Ahmed Hany